

RocketPy: A Six Degree-of-Freedom Rocket Trajectory Simulator

Giovani H. Ceotto¹, Rodrigo N. Schmitt², Guilherme F. Alves³, Lucas A. Pezente⁴, and Bruno S. Carmo⁵

¹Escola Politécnica of the University of São Paulo - Av. Prof. Luciano Gualberto, 380, São Paulo, Brazil, 05508-010 - Email: ghceotto@gmail.com - *Corresponding author

²Institute of Physics of the University of São Paulo - R. do Matão, 1371, São Paulo, Brazil, 05508-090 - Email: rodrigo.schmitt@alumni.usp.br

³Escola Politécnica of the University of São Paulo - Av. Prof. Luciano Gualberto, 380, São Paulo, Brazil, 05508-010 - Email: gf10.alves@gmail.com

⁴Institute of Mathematics and Statistics of the University of São Paulo - R. do Matão, 1010, São Paulo, Brazil, 05508-090 - Email: lucas.pezente@usp.br

⁵Escola Politécnica of the University of São Paulo - Av. Prof. Luciano Gualberto, 380, São Paulo, Brazil, 05508-010 - Ph.D. - Email: bruno.carmo@usp.br

This is a pre-print manuscript. The final published article can be found at: <https://ascelibrary.org/doi/10.1061/%28ASCE%29AS.1943-5525.0001331>

ABSTRACT

RocketPy is a new open-source Python library specialized in trajectory simulations of sounding rockets and high-powered rockets. Having a modular structure and being built on top of a six degree-of-freedom flight dynamics model which includes detailed mass variation effects, RocketPy allows for accurate trajectory prediction of different rocket configurations. Additionally, weather data from several meteorological agencies, such as reanalysis, forecasts and ensembles, are an innovative feature directly integrated into the library, providing precise information about atmospheric conditions for each simulation. The software was successfully validated for three rockets

developed by different universities, with apogee predictions showing deviations of the order of 1% when compared to actual flight data. Furthermore, due to its modular nature, RocketPy can be easily employed in optimization studies and Monte Carlo analyses. The latter was a key aspect of this work, enabling the calculation of landing dispersion ellipses based on input uncertainties. Since running several Monte Carlo analyses is computationally expensive, a new algorithm to significantly speed up results based on rejection sampling was proposed. The code described here and made available online enhances the state of the art represented by several widely used software and can serve as a framework for future research on rocket dynamics, control, flight optimization and dispersion analysis.

INTRODUCTION

Rockets can vary greatly in size and purpose. Between the large rockets responsible for carrying satellites into orbit and small home-made toy rockets, two rocket categories are becoming increasingly similar: sounding rockets and high-powered rockets (HPRs). Sounding rockets were designed during the Second World War mainly as a test-bed for missile programs, and are currently used, among other applications, for scientific experiments in suborbital flights. Most sounding rockets are designed or ordered by government agencies such as NASA (National Aeronautics and Space Administration - USA) and ISRO (Indian Space Research Organisation). HPR, on the other hand, typically refers to the design and construction of custom rockets powered by motors with total impulses up to $900 \text{ kN} \cdot \text{s}$. This activity is legally standardized in several countries by recognized organizations, such as the Federal Aviation Administration (FAA) in the United States.

HPRs have been extensively used as model rockets for educational purposes and have become increasingly popular in student competitions, which occur regularly in both hemispheres. Remarkably, HPR reached new frontiers in 2019 when TRAVELER IV became the first university-built rocket to reach the Kármán line (Aitoumezi et al. 2019). Recent achievements such as this made university rockets increasingly similar to sounding rockets in terms of flight trajectory, since both categories perform sub-orbital flights usually in the supersonic regime with ever closer apogee. This phenomenon was made possible by technological advancements in the design and construction

of HPRs, which have made rockets less expensive and allowed for an increase in scale.

When designing a rocket, it is of utmost importance to have a tool that is able to reliably predict the trajectory of the vehicle given parameters like rocket geometry, motor characteristics and environmental conditions. Predicting apogee altitude, for instance, is important for ensuring the rocket does not exceed permitted safety altitudes and for avoiding collisions with commercial and general aircraft or military fighters. In addition, being able to estimate the landing site coordinates is crucial for both recovery and safety, as it provides *a priori* information regarding possible landing regions (Wilde 2018). Such prediction tool saves time, money, as well as improving operational safety, since experimental tests are restricted to more mature stages of the design. In this paper, RocketPy is presented as a computational tool that is able to accurately predict all dynamic parameters involved in the flight of sounding, model, and high-powered rockets.

Regarding what has been done so far in terms of flight dynamics software, currently there are well established commercial options available such as RockSim (Apogee Rockets 2020), for HPR, and Astos (Astos Solutions GmbH 2018), for sounding rockets, missiles and more complex missions. Open-source or free software such as OpenRocket (Niskanen 2009), RASAero (Rogers and Cooper 2019), or more recent ones such as the Cambridge Rocketry Simulator (Box et al. 2011; Eerland et al. 2017) are available as well. However, these are generally used only by university groups and are in consequence focused on HPR, having limited applicability to other types of vehicles. It is also worth noting that algorithms designed to simulate sounding rocket trajectories are rather common in the literature, such as the one described in Da Silveira and Carrara (2015). These are usually tailored to rather specific missions that require a different set of functionalities from what is commonly found in most open-source codes.

One of the most used flight simulation software nowadays in model rocketry is OpenRocket, an open-source solution using a six degree-of-freedom flight model for the rocket ascent and a three degree-of-freedom flight model for parachute descent. This software allows users to input various geometric properties of the rocket to produce a simulation of the entire flight using semi-empirical aerodynamics (Niskanen 2009). Nevertheless, it only provides results for a specific set of flight

conditions, which is far from ideal as most input data contain significant uncertainties.

On top of that, Box et al. (2011) and Eerland et al. (2017) introduced a method to perform stochastic simulations using the Cambridge Rocketry Simulator (CRS), which treats wind as a function of altitude and with a stochastic probabilistic approach. CRS, like OpenRocket, is also based on a six degree-of-freedom model for the rocket flight and three degree-of-freedom model for parachute descent. Even though the Monte Carlo method is applied, only a few input parameters are varied for the stochastic analysis, leaving out the rocket's inertia and geometric parameters which can significantly reduce the confidence in the flight analysis.

When designing RocketPy, the main goal was to develop a tool to be available and useful for as many applications regarding HPR simulation as possible. To achieve this goal, the software was conceived as an open-source platform and all its methods were implemented without significant simplifications, or at least the simplifications were isolated and clearly identified. The chosen dynamical flight model was complete enough to be capable of handling any flight using a 6-DOF description, just like OpenRocket. In addition, the atmospheric conditions can easily be imported from several specialized agencies' datasets, allowing for faster and more accurate wind profile calculations when compared to both OpenRocket and CRS. The Monte Carlo method is also available in RocketPy, as it is in CRS, but this time allowing for simulations varying every desired input, and using any desired probability distribution. More importantly, RocketPy's code is well structured and documented, allowing future users to easily contribute and improve the current code base. Being a very flexible tool, minor changes in the source-code can allow RocketPy to have new sophisticated features. This flexibility and easiness make RocketPy a powerful and competitive software, since it allows for faster and larger development of the tools over the years, even when its features start to be created from some inspirations provided by other software. Table 1 presents a summary of RocketPy's distinctive points. All these features make RocketPy a software that can simulate the flight of different rockets, using both innovative and already-known features.

In what follows, we first present an overview of the internal algorithmic structure of the software in section 2. Explanations regarding the simulation models, which are embedded into the code, are

TABLE 1. Comparison of RocketPy with previous works.

Feature	OpenRocket (Niskanen 2009)	Cambridge Rocketry Simulator (Eerland et al. 2017)	RocketPy
Programming Language	Java	Java	Python
Open Source	x	x	x
3-DOF Parachute Descent	x	x	x
6-DOF Ascent	x	x	x
Euler Parameters Output		x	x
Monte Carlo Dispersion Analysis		Limited to a few parameters	x
Reanalysis, Forecasts and Ensembles			x
Multivariate Rejection Sampling			x

thoroughly discussed together with the given physical assumptions towards the phenomena at hand. Next, in section 3 more explanation is given in regards to solving the equations of motion and how data flows within the internal structure of the program as to produce fast and precise simulations. Additionally, specific information regarding the numerical integration scheme is further detailed. In section 4, simulation results are first verified against other well established software in the market. Next, comparisons against real data obtained from sounding rockets recently launched are made and key parameters regarding the flight dynamics are analyzed. In addition, the Monte Carlo Method is discussed within the context of dispersion analysis, as well as an statistical approach named Multivariate Rejection Sampling (MRS). Finally, in section 5 conclusions are drawn.

ROCKETPY ARCHITECTURE

RocketPy was implemented using the interpreted language Python (Van Rossum and Drake 2009) and some of its scientific libraries, which include *NumPy* (Harris et al. 2020) and *SciPy* (Virtanen et al. 2020). Other languages such as Julia, MATLAB, C and C++ were considered for the implementation, but Python was chosen for its simplicity, flexibility and increasing popularity in academia and in the industry. These factors facilitate the dissemination and integration of RocketPy. So far, the main interface used to run RocketPy simulations has been Jupyter Notebooks (Kluyver et al. 2016); however, other methods are also possible.

RocketPy is divided into four main classes to organize data flow throughout a simulation. These are related as shown in Figure 1. First, the configuration of a Solid Motor is defined: the nozzle

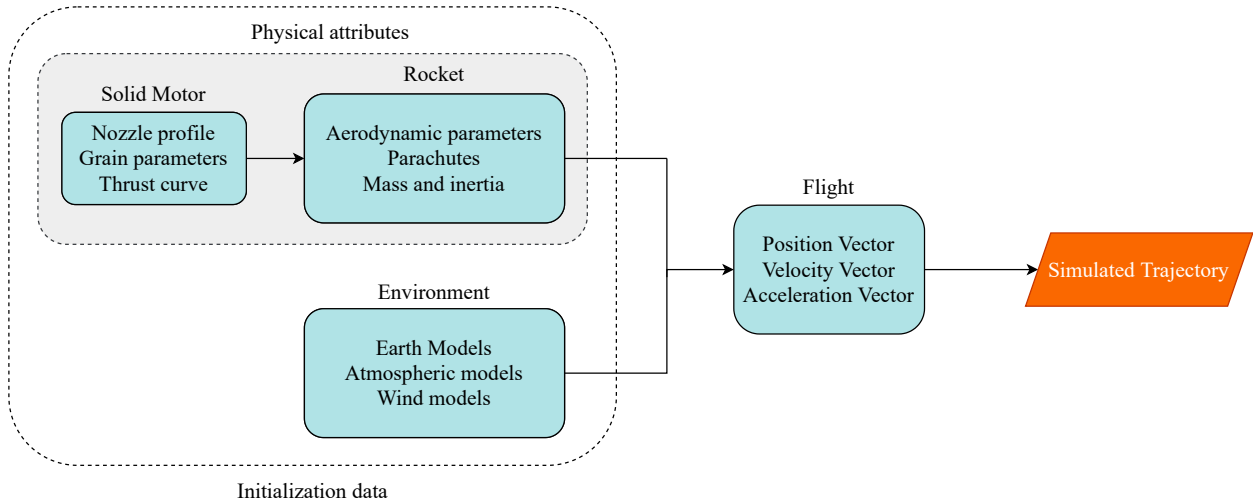


Fig. 1. RocketPy Architecture.

geometry, grain parameters, mass, inertia and thrust curve. The Solid Motor object is an input to the Rocket class, which also requires the user to define some of the rocket's parameters, such as the mass and inertia tensor, geometry, aerodynamic coefficients and parachute description. Together with information from the Environment class, such as wind, atmospheric and Earth models, the rocket and motor parameters are then used by the Flight class to simulate the rocket's trajectory.

SIMULATION MODELS AND METHODS

Each of the four main classes presented in section 2 employs specific methods and simulations models to convert user inputs into the desired outputs. These models and methods are described next, emphasizing the main features and limitations of the proposed system.

Environment Model

The purpose of the Environment class is reading, processing and storing basic information regarding the launch date, latitude and longitude of the launch site, atmospheric conditions, and Earth model. Besides the launch site's elevation, which comes from the Earth model, it supplies the Flight class with six profiles as a function of altitude: wind velocity in both east and north directions, atmospheric pressure, air density, dynamic viscosity and speed of sound.

In terms of atmospheric models, the International Standard Atmosphere (ISO Central Secretary

1975) is implemented by default. However, RocketPy offers direct integration with real weather data, including inputs from important meteorological agencies such as the European Centre for Medium-Range Weather Forecasts (ECMWF), the Canadian Meteorological Centre (CMC) and the National Oceanic and Atmospheric Administration (NOAA) through netCDF files or Open-source Project for a Network Data Access Protocol (OPeNDAP). Soundings are also supported, including integration with NOAA's Ruc Sounding servers and University of Wyoming's soundings.

These contain all necessary information on the state of the atmosphere for several years before the flight. Therefore, atmospheric conditions for a certain trajectory simulation may be well represented by using the wind condition from a day in the same season in previous years. For example, if a rocket launch is expected to happen on January 8th, 2021, weather data from several days near January 8th, from 2010 to 2020, may be used.

It is worth noting that RocketPy also supports ensemble forecasts, a kind of numerical weather prediction method which takes into account different kinds of uncertainties associated with the mathematical models and the nature of the phenomena. This kind of forecast relies on Monte Carlo analysis, which generates a set of possible states that on average should give a good indication of the possible future scenarios. This is very different from reanalysis, as it also accounts for previous information but does not take into consideration model inadequacies or the nature of the phenomena.

Given an atmospheric model, which is defined by the pressure, temperature, wind velocity in both east and north directions and surface elevation as a function of altitude, the Environment class is used to compute the following parameters: wind speed, wind direction (from which the wind blows), wind heading (towards which the wind blows), speed of sound, air density and dynamic viscosity. Plots of the wind profiles, air density, speed of sound, pressure and temperature with respect to the altitude are all given to the mission analyst.

Earth models can be imported in a similar fashion, for example using OPeNDAP protocol to import topographic data for NASA's Digital Elevation Models. In the present work, the Earth model used was the simple flat Earth, given that low apogee flights did not benefit from a more

complex model which would come with a greater processing time.

Solid Motor Model

Various types of rocket motors exist and RocketPy is flexible enough to work with most of them. Currently, a Solid Motor class has been fully implemented and tested, considering sounding rockets generally use solid motors. The main functionality of the motor class is to provide the produced thrust, propellant mass, inertia tensor, and the position of its center of mass as a function of time.

To accomplish this, RocketPy's Solid Motor class takes as input a static test thrust curve, which is commonly available for commercial motors. The geometry of the propellant grains is also fully defined, along with its density. These inputs can be used together to calculate the variation of propellant mass and inertia tensor during the motor's burn. To do so, precise information about the propellant burn rate would be necessary. Since this is rarely available, RocketPy assumes constant specific impulse, or constant exhaust velocity, instead. With this assumption, the exhaust velocity and the propellant mass are respectively given by

$$v_e = \frac{I_{tot}}{m_p(0)}, \quad \dot{m}_p(t) = -\frac{\|\mathbf{T}(t)\|}{v_e}, \quad (1)$$

where I_{tot} is the motor's total impulse, $m_p(t)$ is the propellant mass as a function of time, and $\|\mathbf{T}(t)\|$ is the magnitude of motor's thrust as a function of time. Pressure difference terms are neglected since a perfectly expanded nozzle is considered. The variation of propellant mass can then be used to compute the variation of propellant grain geometry and thus its inertia tensor as a function of time by solving a set of differential equations numerically, as shown in Panicucci et al. (2018).

Rocket Model

The Rocket class holds, processes, and supplies the main dynamics and aerodynamics characteristics of the rocket itself. While other software usually calculate rocket's inertia tensors by themselves, RocketPy imports these data directly from specialized software or experimental data, instead of calculating them based on the rockets geometry and materials. In terms of aerodynamic

coefficients, the same reasoning was employed. Wind tunnel data, computational fluid dynamics analysis or other resources can be used to determine the aerodynamic characteristics of a rocket.

RocketPy focuses on importing and processing these data adequately to supply them to the equations of motion as needed. An exception to this is the calculation of the lift coefficients of common rocket lifting surfaces, such as nose cones, fins and tails. Even though external data can still be used, RocketPy implements Barrowman's method (Barrowman 1967) to calculate the lift coefficient and the center of pressure of each lifting surface.

This class also takes the rocket's motor as input, which allows for the calculation of the total mass and inertia tensor of the rocket as a function of time. In addition, parachutes are also configured in this class. The user is free to specify as many parachutes as needed by setting their parameters such as opening delay, drag coefficients, and trigger functions.

Trigger functions take noise added pressure data from the flight simulation as input and return a boolean variable which informs if the parachute should be deployed or not. RocketPy allows the user to specify a sampling rate for each trigger, augmenting the behavior of a real sensor. The dense output from the integrator is used every few iterations, supplying the trigger function with data interpolated between time-steps, to make sure the sampling rate is respected. This allows for a faster and more accurate simulation which may use variable time-step sizes rather than fixed time-steps to comply with a parachute trigger function. These triggers may be written in any language that can interface with Python, which permits the actual parachute activation code used during the flight to be called in the simulation.

Flight Model

A sounding rocket flight can be divided in at least three different flight phases. The first phase is characterized by a linear motion along the launch rail. Once the rocket is completely free from the rail, a six degree-of-freedom phase is established, which includes powered flight and free flight. After the apogee is reached, a parachute is usually deployed, characterizing the third flight phase: descent under parachute. This section presents the equations of motion used in RocketPy for each of these flight phases.

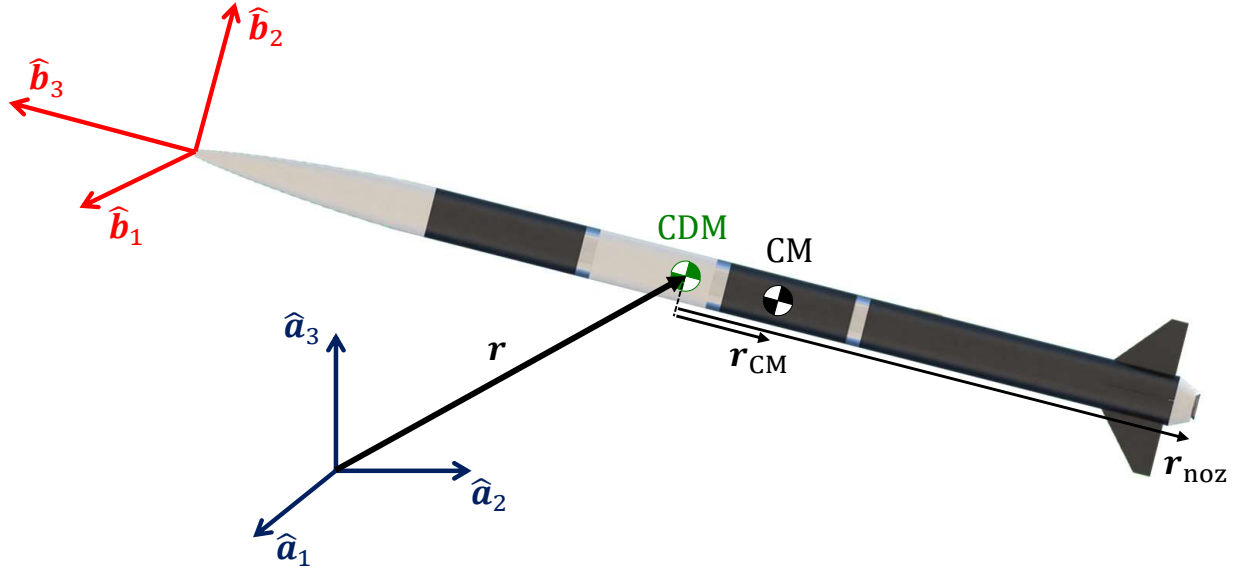


Fig. 2. Coordinate systems used in the Flight Model.

Two reference frames will be used throughout this section. Fixed to the rigid body of the rocket is the coordinate system $\mathcal{B}$, whose axes are oriented considering a right-handed orthogonal base $b_1 b_2 b_3$, with $\hat{b}_3$ aligned with the rocket's axis. The second coordinate system, $\mathcal{A}$, is fixed to Earth, which is assumed inertial, and also oriented by a right-handed orthogonal base $a_1 a_2 a_3$. Both frames are shown in Figure 2, which also displays the Center of Mass (CM) of the complete rocket, the Center of Dry Mass (CDM), which does not take the propellant mass into account and the Center of Propellant Mass (CPM).

Throughout the trajectory simulation, the rocket's state vector $\mathbf{u}$ will be tracked and calculated. Mathematically, $\mathbf{u}$ is defined by:

$$\mathbf{u} = \begin{bmatrix} \mathbf{r} & \mathbf{v} & \mathbf{q} & \boldsymbol{\omega} \end{bmatrix}^T \quad (2)$$

where $\mathbf{r} = [x, y, z]_{\mathcal{A}}$ is the position vector of the rocket's center of dry mass, relative to its initial position, and $\mathbf{v}$ is the time derivative of $\mathbf{r}$. The quaternion $\mathbf{q}$ is used to represent the rocket's attitude in terms of Euler parameters, avoiding the singularities present when Euler angles are employed. The rocket's angular velocity is denoted by $\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]_{\mathcal{B}}$. The time derivative of the attitude

quaternion is related to ω by:

$$\dot{\mathbf{q}} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_1 & -\omega_2 & -\omega_3 \\ \omega_1 & 0 & \omega_3 & -\omega_2 \\ \omega_2 & -\omega_3 & 0 & \omega_1 \\ \omega_3 & \omega_2 & -\omega_1 & 0 \end{bmatrix} \mathbf{q} \quad (3)$$

This means that two of the four quantities of the rocket's state vector $\mathbf{u}$ have a known derivative expression: $\mathbf{r}$ and $\mathbf{q}$. Nonetheless, in order to fully characterize the dynamic model, the derivatives of $\mathbf{v}$ and ω , which vary depending on the current flight phase, are also needed. In the following sub-sections, these are presented for each of the three flight phases.

Motion in the Launch Rail

When moving along the launch rail, a sounding rocket has its motion restricted to one dimension, which means only translation along the rail needs to be modeled. During this phase, four forces can act on the rocket: weight, thrust from the motor, reactions from the rail and aerodynamic forces. The thrust force is a function of time and denoted by $\mathbf{T} = T(t)\hat{\mathbf{b}}_3$, with $T(t)$ supplied by the user through the Solid Motor class. Reactions from the rail in the direction of the rail are usually negligible. Aerodynamic forces acting on rockets are usually decomposed into the axial force $\mathbf{A}$ and the normal force $\mathbf{N}$. Since the normal force $\mathbf{N}$ is perpendicular to the rocket's axis, which is aligned to the rail, its component in the rail direction, $\hat{\mathbf{b}}_3$, is zero. The axial force $\mathbf{A}$ is modeled by:

$$\mathbf{A} = \frac{1}{2} C_D \rho \|\mathbf{V}_\infty\|^2 S (-\hat{\mathbf{b}}_3) \quad (4)$$

where the drag coefficient C_D is a function of Mach number and is supplied by the user through the Rocket class. $\mathbf{V}_\infty$ denotes the relative velocity of the rocket with respect to the wind:

$$\mathbf{V}_\infty = \mathbf{W}(\mathbf{r}) - \mathbf{v} \quad (5)$$

where the wind velocity $\mathbf{W}$ is a function of the rockets position vector, mainly its altitude.

Combining the forces acting on the rocket, the time derivative of $\mathbf{v}$, the rocket's center of dry

mass acceleration is given by:

$$\underbrace{m(\mathbf{v} + \mathbf{r}_{\text{CM}}'')}_{\text{CM acceleration}} = \underbrace{\mathbf{T} - 2\dot{m}\mathbf{r}_{\text{CM}}'}_{\text{Mass variation terms}} + \underbrace{\ddot{m}(\mathbf{r}_{\text{noz}} - \mathbf{r}_{\text{CM}}) + \mathbf{A} - (mg\hat{\mathbf{a}}_3 \cdot \hat{\mathbf{b}}_3)\hat{\mathbf{b}}_3}_{\text{External forces}} \quad (6)$$

where $\mathbf{r}_{\text{CM}}$ is the position vector of the rocket's center of mass relative to its center of dry mass and $\mathbf{r}_{\text{noz}}$ is the position vector of the rocket's nozzle exit also relative to the center of dry mass. Their derivatives are represented by the prime notation, indicating that they are relative to the coordinate system attached to the rocket's body, $\mathcal{B}$. The variable m is a function of time and represents the varying mass of the rocket, which is computed by adding the rocket's dry mass to the propellant mass. To complete the model for this flight phase, the rocket's attitude is kept constant and thus $\dot{\omega} = \omega = 0$.

Six Degree-of-Freedom Flight

Once the rocket has cleared the launch rail, it is free to move in three-dimensional space, starting a six degree-of-freedom flight. Weight, motor thrust, axial and normal aerodynamic forces are still important. The first three of these obey the same mathematical model used in the rail phase.

On the other hand, to compute the normal force $\mathbf{N}$, RocketPy subdivides the rocket into lifting surfaces. These usually include the nose cone, diameter variations, fin sets and tail. Custom surfaces may also be added. Consider the i -th lifting surface whose center of pressure position vector relative to the rocket's center of dry mass is denoted by $\mathbf{r}_i$. Its center of pressure moves with velocity $\mathbf{v}_i = \mathbf{v} + \omega \times \mathbf{r}_i$. Therefore, the relative velocity of the wind with respect to the center of pressure of the i -th lifting surface is:

$$\mathbf{V}_{\infty,i} = \mathbf{W}(\mathbf{r} + \mathbf{r}_i) - \mathbf{v} - \omega \times \mathbf{r}_i \quad (7)$$

where the last term, $-\omega \times \mathbf{r}_i$, will result in an oscillation damping term for each lifting surface. The angle of attack α_i which this lifting surface will experience is defined as the angle between its relative velocity vector and the rocket's axis. Mathematically, it can be written as:

$$\alpha_i = \cos^{-1} \left(\frac{-\mathbf{V}_{\infty,i} \cdot \hat{\mathbf{b}}_3}{\|\mathbf{V}_{\infty,i}\|} \right) \quad (8)$$

With the relative velocity $\mathbf{V}_{\infty,i}$ and the angle of attack α_i defined, it is possible to calculate the normal aerodynamic force $\mathbf{N}_i$ which acts on this lifting surface. Assuming a lift coefficient $C_L(\alpha_i)$ given by the Rocket class, the normal force $\mathbf{N}_i$ is given by:

$$\mathbf{N}_i = C_L(\alpha_i) S \left(\frac{1}{2} \rho \|\mathbf{V}_{\infty,i}\|^2 \right) \left(\hat{\mathbf{b}}_3 \times \frac{\mathbf{V}_{\infty,i}}{\|\mathbf{V}_{\infty,i}\|} \right) \quad (9)$$

Another crucial aspect needed to establish the equations of motion is an appropriate description of the forces and moments which arise from variable mass effects, which was based on Eke and Mao (2002). To remove certain simplifying assumptions that may not be adequate for certain rockets, the methodologies proposed by Thomson (1966) and Panicucci et al. (2018) were also used. The remaining simplifying hypothesis are:

- Liquids are not present or sloshing can be neglected;
- The motor operates in steady-state and transient effects are negligible;
- Exhaust gases have a uniform velocity profile at the nozzle's exit, without swirl.

Therefore, accounting for variable mass effects and having a known expression for weight, thrust, drag and normal aerodynamic force for every lifting surface i , the equation of motion for the rocket's center of dry mass can be expressed by:

$$\underbrace{m (\mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}_{\text{CM}} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r}_{\text{CM}}) + \mathbf{r}_{\text{CM}}'' + 2\boldsymbol{\omega} \times \mathbf{r}_{\text{CM}}')}_{\text{CM acceleration}} = \underbrace{\mathbf{T} - 2\dot{m}\mathbf{r}_{\text{CM}}' + 2\boldsymbol{\omega} \times \dot{m}(\mathbf{r}_{\text{noz}} - \mathbf{r}_{\text{CM}}) + \ddot{m}(\mathbf{r}_{\text{noz}} - \mathbf{r}_{\text{CM}})}_{\text{Variable mass effects}} + \underbrace{\mathbf{A} + \sum_i \mathbf{N}_i - mg\hat{\mathbf{a}}_3}_{\text{External forces}} \quad (10)$$

Here, the left-hand side represents mass times acceleration of the true center of mass of the rocket, while the right-hand side is a sum of forces arising from mass variation and external forces: weight and aerodynamic forces (axial and normal). Equation (10) is well defined as long as m , $\mathbf{r}_{\text{CM}}$ and their derivatives are known. These variables are calculated in the Rocket class and are given as functions of time.

The time derivative of the angular velocity can be calculated considering the moments due

to the normal forces, along with jet damping (Thomson and Reiter 1965) and gyroscopic effects, assuming the CDM and the CPM are aligned, by:

$$\underbrace{\mathbf{I} \cdot \boldsymbol{\omega} + \boldsymbol{\omega} \times (\mathbf{I} \cdot \boldsymbol{\omega}) + \mathbf{I}' \cdot \boldsymbol{\omega}}_{\text{Angular momentum derivative}} + \underbrace{m\mathbf{r}_{\text{CM}} \times \mathbf{v}}_{\text{Inertial moment}} = \underbrace{(\dot{m}\mathbf{S}_{\text{noz}}) \cdot \boldsymbol{\omega}}_{\text{Jet damping}} + \underbrace{\sum_i \mathbf{r}_i \times \mathbf{N}_i - \mathbf{r}_{\text{CM}} \times mg\hat{\mathbf{a}}_3}_{\text{External moments}} \quad (11)$$

In this equation, $\mathbf{S}_{\text{noz}}$ is a geometric parameter of the nozzle that denotes the gyration tensor of an imaginary surface corresponding to the nozzle's exit area and $\mathbf{I}$ is the rocket's inertia tensor calculated relative to the center of dry mass, which is assumed diagonal in RocketPy, meaning products of inertia are neglected. It is worth emphasizing that $\mathbf{I}'$, the time derivative of $\mathbf{I}$, is computed relative to the body frame $\mathcal{B}$, so it is calculated simply as the derivative of each of its scalar elements. Both inertia tensors are given by the Rocket class as functions of time.

Equations (10) and (11) can be used to calculate $\mathbf{v}$ and $\boldsymbol{\omega}$. Therefore, the derivative of the state vector $\mathbf{u}$ is fully specified and can be integrated to calculate the evolution of the rocket's position and attitude over time. The numerical integration process occurs until an event is triggered, such as the ejection of a parachute.

Descent Under Parachute

When a parachute is ejected from a sounding rocket, the rocket is usually divided into two or more parts attached by ropes. In this scenario, multi-body dynamics is present during descent. This is further complicated by the dynamics of parachute inflation, which often requires several empirical parameters to be well characterized. However, many details of the full dynamics of descent under parachute are often not important. The main information needed from this flight phase is the terminal vertical velocity attained by the system, along with the horizontal velocity imposed by lateral drag due to the wind. Therefore, RocketPy implements a significantly simplified model which represents both of these requirements well. The model begins with the assumption of instantaneous inflation. In addition, all rocket parts, including the parachute, are concentrated into a single material point whose motion will be determined by two forces: parachute drag and weight.

The drag force, $\mathbf{D}$, is calculated considering the free stream velocity as defined by Eq. (5):

$$\mathbf{D} = \left(\frac{1}{2} \rho \|\mathbf{V}_\infty\|^2 \right) C_D S \hat{\mathbf{V}}_\infty \quad (12)$$

where $C_D S$ is the product of the active parachute's drag coefficient and reference area. This leads directly to the equation of motion:

$$\mathbf{v} = -\frac{1}{m} \left(\frac{1}{2} \rho(\mathbf{r}) \|\mathbf{V}_\infty\|^2 \right) C_D S \hat{\mathbf{V}}_\infty - g \hat{\mathbf{a}}_3 \quad (13)$$

Since this is a material point model, the derivative of the angular velocity is kept null.

Numerical integration

The numerical integration scheme LSODA, developed by Petzold (1983) and Hindmarsh (1992), was employed for the solution of the differential equations resulting from the models presented above. LSODA can solve both stiff and non-stiff equations efficiently. In the case of non-stiff equations, typical of a stable rocket flight, LSODA employs a variable order, up to 13th order, variable time step Adams-Moulton predictor-corrector scheme. When the rocket flight is unstable, the equations of motion may become stiff and, in such case, LSODA automatically employs Backward Differentiation Formula methods.

The block diagram for the numerical integration is shown in Figure 3. The rocket's state vector runs through the integrator using the equations of motion of the current flight phase – $\dot{\mathbf{u}}_R$ for the Rail Phase, $\dot{\mathbf{u}}_{FF}$ for the Free Flight Phase or $\dot{\mathbf{u}}_P$ for the Parachute Phase – and eventually terminates when the rocket hits the ground. A check is performed at every iteration to verify if a change of flight phase has occurred. For instance, parachute trigger functions are fed with the atmospheric pressure experienced by the rocket at every time instant and their response determines if a parachute should be deployed, in which case the parachute flight phase is activated. Event location methods are used for rail departure, apogee, and impact events to assure accuracy.

RESULTS

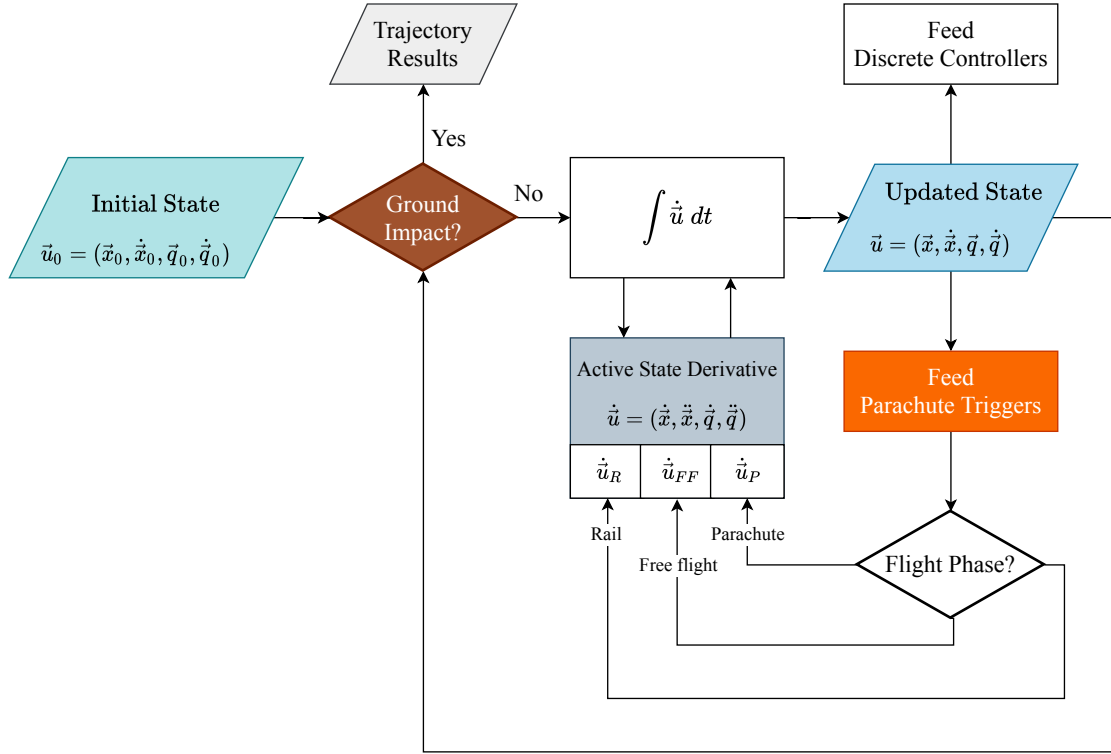


Fig. 3. Integrator Architecture.

Comparison with previous works

An example flight simulation performed with RocketPy is now presented, considering the launch vehicle named Valetudo, an Experimental Sounding Rocket developed by the Projeto Jupiter team from the University of São Paulo. Valetudo is an 80 mm diameter, 2.4 m long rocket, weighing 9.67 kg when loaded and equipped with a class K solid motor.

Figure 4(a) shows a comparison of Valetudo's flight simulation results obtained from three different software: RocketPy, OpenRocket and Cambridge Rocketry Simulator (CRS). For RocketPy, atmospheric conditions were selected directly from weather ensemble reanalyses, as shown in Figures 4(e) and 4(f). These data were exported from RocketPy and used as a custom atmosphere in the CRS analysis. Finally, a few discrete values were used as inputs in OpenRocket, such as the average of wind direction.

The flight starts at the launch pad at coordinates equal to (0, 0, 0), which represents an elevation

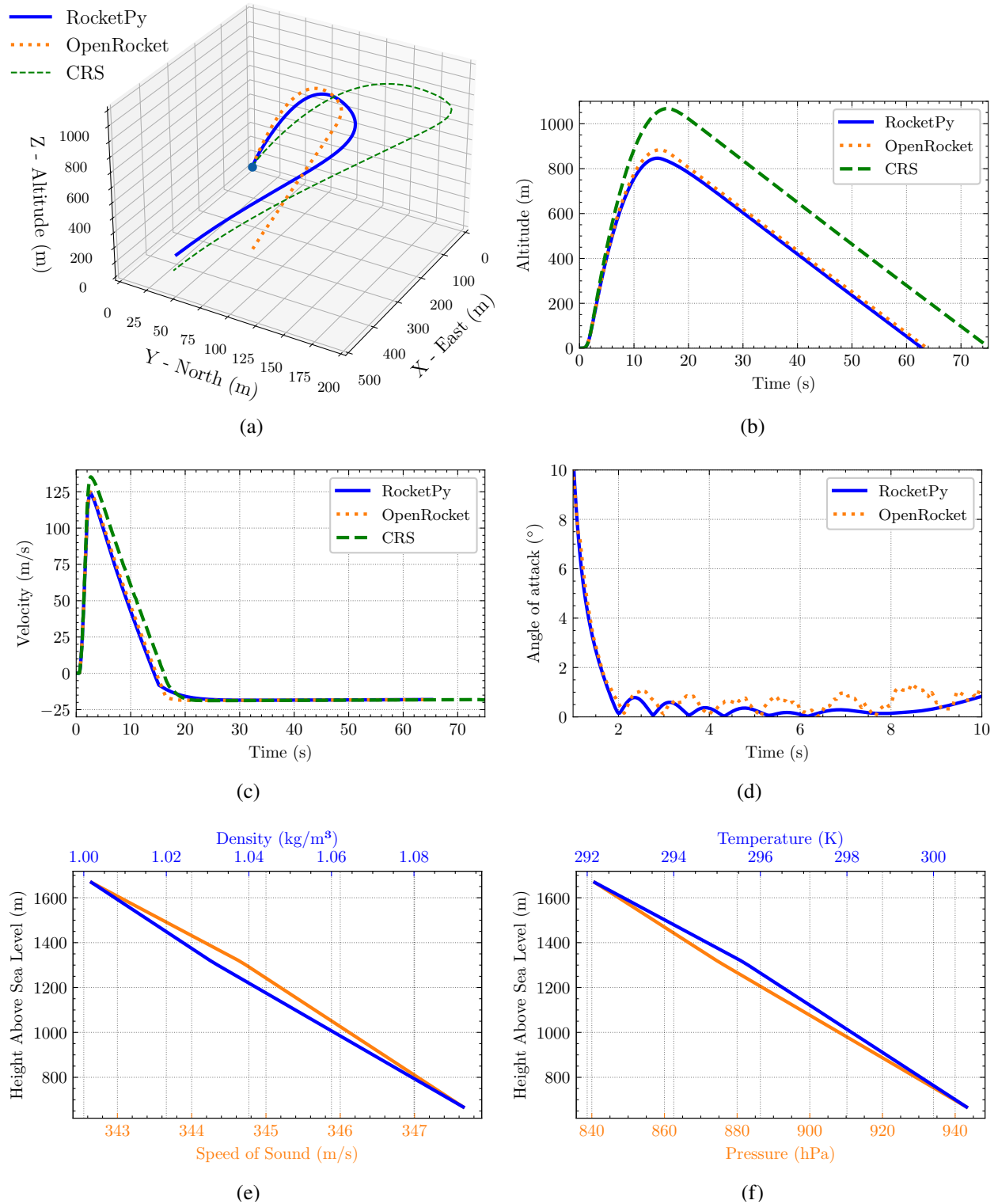


Fig. 4. Altitude and vertical velocity as a function of time obtained from Valetudo flight simulation with RocketPy, OpenRocket and Cambridge Rocketry Simulator (CRS): (a) Trajectory, (b) Altitude, (c) Vertical velocity, (d) Angle of attack, (e) Speed of sound and density profile, and (f) Pressure and Temperature profile.

TABLE 2. Valetudo flight simulation results obtained with RocketPy, OpenRocket and Cambridge Rocketry Simulator.

Result parameter	RocketPy	OpenRocket	CRS
Apogee altitude (m)	846.65	882.42	1067.82
Apogee time (s)	14.29	14.73	16.03
Impact vertical velocity (ms^{-1})	18.08	18.14	18.15
Maximum vertical velocity (ms^{-1})	124.02	124.87	135.26

of 668 m above sea level. During the ascent, Valetudo drifts horizontally as it rises to around 900 m. Right after the apogee, a drogue parachute is deployed and the rocket is projected back into the wind as it descends, then landing 350 m east and 25 m north of the launch pad. Valetudo was equipped with a barometric altimeter named Rocket Recovery Controller 3 (RRC3), which registered an apogee altitude of 860 m above ground level.

These motion data are also shown in two dimensional plots in Figure 4(b) and Figure 4(c). It is evident that all the altitude curves start with a similar growth until around 5 seconds, when the CRS continues to a higher apogee altitude and drift. It can be noticed, however, that this curve's slope comes to be equal to the other two in $t = 20$ s, when the parachute in the simulation opens, as presented in the vertical velocity graph. This is a strong hint that the three simulations are closely related, even though CRS is slightly off during a significant time interval.

Figure 4(d) shows that the angles of attack simulated by both RocketPy and OpenRocket converge in the beginning but diverge after they first reach zero. This is a result of the different methods used by RocketPy and OpenRocket to model oscillation damping. While OpenRocket considers an overall pitch damping coefficient, RocketPy considers the damping caused by each lifting surface according to its local free stream velocity as shown in equation 7. Furthermore, numerical integration accuracy also manifests itself here. OpenRocket uses a fixed-step, fourth-order, Runge-Kutta algorithm while RocketPy employs the variable-step, variable-order, LSODA algorithm as explained in section 3, providing a more accurate and smooth response.

Table 2 presents a brief summary of the results regarding the flight of Valetudo.

Experimental Results

Comparison plots and strategic parameters were chosen to verify the validity of RocketPy simulations. Data collected from experimental rockets avionics were analyzed for the comparison to be made. First, an analysis concerning the École Polytechnique Fédérale de Lausanne Rocket Team (ERT) Bella Lui Kaltbrunn mission (2020) is made. The rocket Bella Lui was propelled by a class K solid motor with a total impulse of $2120 \text{ N} \cdot \text{s}$.

Figure 5(a) shows the altitude of the rocket as a function of time where a very close match can be seen between predicted data and real measurements up until apogee. The good agreement shown in this first flight phase can be explained in different manners. First of all, as previously mentioned, RocketPy imports some of the data used in the simulation from other specialized software, a feature that provides reliable data to critical flight parameters (e.g., aerodynamic coefficients), therefore enhancing the confidence on the simulation output. Furthermore, the environment model earlier described yields precise information regarding the launch site, which allows the simulations to account for realistic atmospheric conditions and wind models.

After the maximum height is reached, a different flight phase is detected by the numerical integrator, as shown in Figure 3, when one parachute is triggered and finds itself fully inflated shortly after. This imposes different dynamics and demands extra assumptions and calculations to be made within the simulator numerical integrator, resulting in significant deviations from the actual data in this particular case. More specifically, assumptions such as instantaneous inflation, although not too simplistic, may be a contributing factor to the differences seen mainly after apogee. Finally, semi-empirical parameters such as the parachute drag coefficient and reference area are hard to be characterized with accuracy, therefore introducing more uncertainty into the calculations.

With respect to the vertical velocity measurements as a function of time, as shown in Figure 5(a), there was a near match between the simulated and measured data. The exception was the velocity during descent with the aid of parachutes, which could also be explained by the oscillations imposed by the parachute's instabilities, which is a factor the current model does not take into consideration.

Next, the flight from NASA's 2020 Student Launch of Notre Dame Rocketry Team (NDRT) was

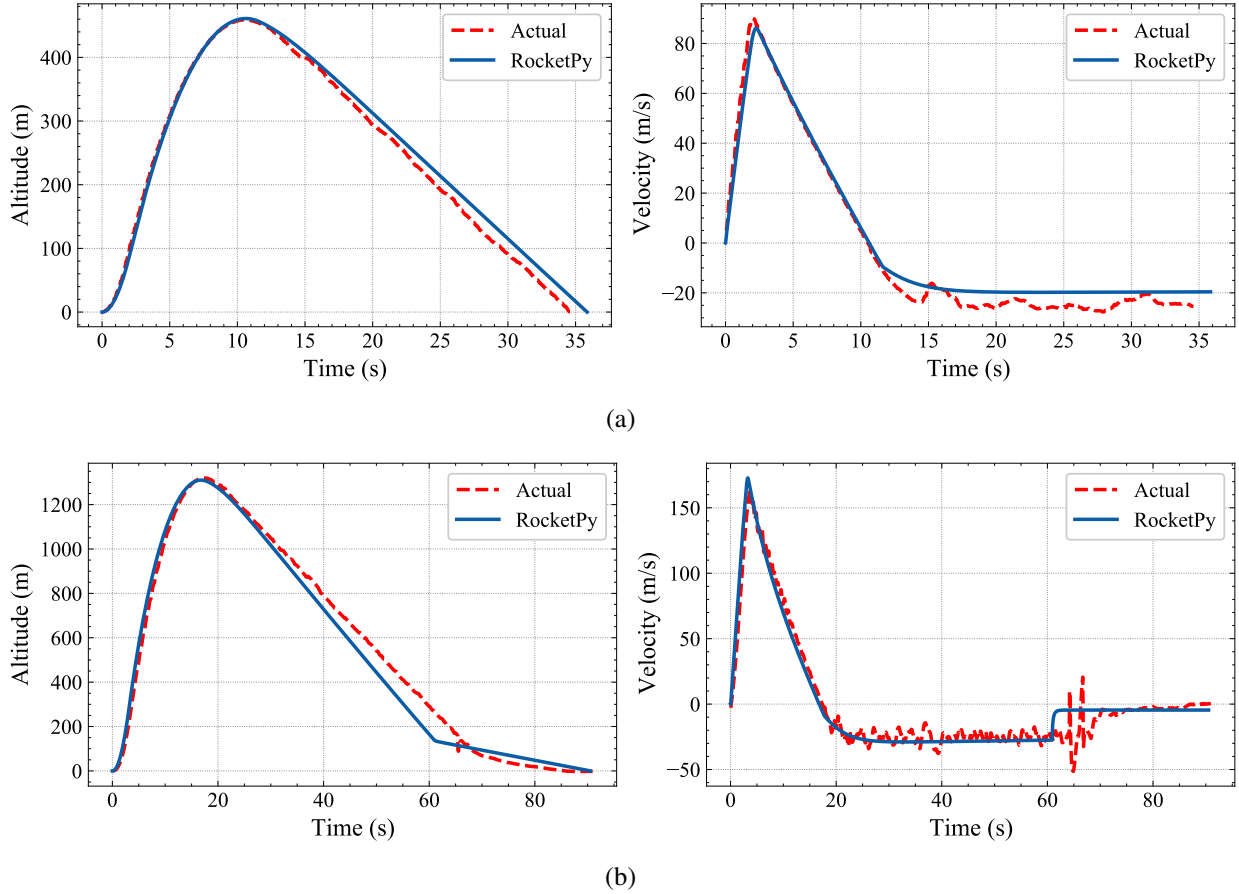


Fig. 5. Real flight data vs simulation results: (a) Comparison for the Bella Lui Kaltbrunn Mission, and (b) Comparison for the 2020 NDRT launch vehicle

simulated and also compared with actual flight data. The causes for disagreement between data and simulated results in the present flight are due to the same causes that have already been discussed for the École Polytechnique Fédérale de Lausanne Rocket Team (ERT) Bella Lui Kaltbrunn mission (2020), with only a few additions. Detailed information about the 3.4 m tall, 24.5 kg vehicle, as well as about its two parachutes, can be found in the mission technical report (Notre Dame Rocketry Team 2020).

Figure 5(b) shows the altitude of the vehicle as a function of time. Overall, a good fit can be seen between simulation outputs and data. The increasing difference that appears after approximately 20 seconds of flight is also due to parachute oscillations, a factor which has already been discussed. Additionally, differences such as that presented at the end of the trajectory and throughout the flight

TABLE 3. RocketPy simulated vs. measured parameters for the Bella Lui Kaltbrunn mission and NDRT launch vehicle.

Mission	Result parameter	RocketPy	Measured	Relative Error
Bella Lui Kaltbrunn	Apogee altitude (m)	461.03	458.97	0.45%
	Apogee time (s)	10.61	10.56	0.47%
	Maximum velocity (ms^{-1})	86.18	90.00	4.24%
NDRT launch vehicle	Apogee altitude (m)	1310.44	1320.37	-0.75%
	Apogee time (s)	16.77	17.10	-1.90%
	Maximum velocity (ms^{-1})	172.86	168.95	2.31%

may also be explained by noting that telemetry equipment has its limitations, such as sampling rate constraints, and it is subject to severe vibrations, which may result in distorted data collection. Nevertheless, the fact that both comparisons presented a good agreement for the ascent flight phase, which is actually the only flight phase considered as 6-DOF motion in the present work, indicates the consistency of the developed model which has been presented in section 3. Unfortunately, experimental data regarding three-dimensional position, velocity and attitude profiles were not available at the time, thus no further discussion concerning the comparison of such parameters will be made in the present section. An indirect three-dimensional comparison will be possible in the next section while presenting Monte Carlo Simulations for dispersion analyses, especially when discussing the impact coordinates results for the Valetudo flight.

Table 3 summarizes the results presented in Figure 5, for both ERT and NDRT launches. The agreement between simulation results and measured data is remarkable.

The Monte Carlo Method

RocketPy has been built from the ground up with the intent of using Monte Carlo analyses to perform statistical estimates of key flight outputs, considering the inputs as uncertain. This generally requires a significant number of flight simulations to yield accurate results. Therefore, the code has been highly optimized to run a full simulation in about a second or less.

The flowchart presented in Figure 6 illustrates how RocketPy can be involved by a wrapper to allow for Monte Carlo analyses. In each iteration of the flowchart loop, the inputs necessary

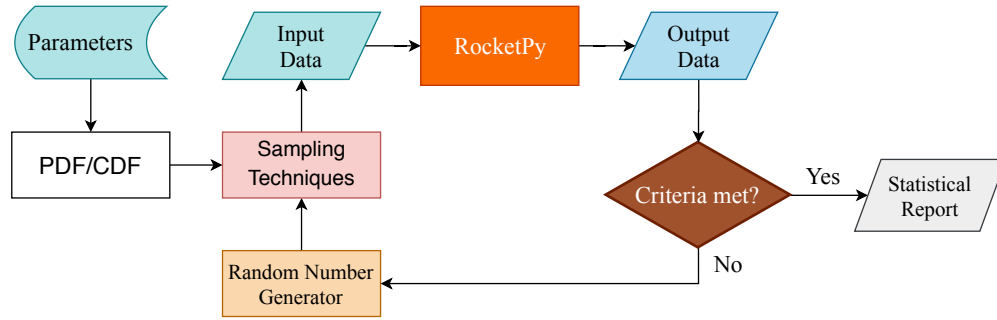


Fig. 6. Monte Carlo-based dispersion analysis.

for RocketPy are generated by sampling techniques which consider the user supplied Probability and Cumulative Density Functions (PDF and CDF) of each variable. The flight trajectory is then simulated, producing relevant outputs such as apogee altitude, impact point, and maximum speed. Subsequently, output confidence intervals may be calculated using bootstrapping methods to determine if the data allows for precise estimation of population means or other user desired statistics. This serves as convergence criteria, determining if the process is looped or terminated.

Since almost every input presents some kind of uncertainty, except for the number of fins or propellant grains that a rocket has, these can be taken into account by RocketPy when Monte Carlo analyses are used. The value of some inputs, such as the rocket's mass, are expected to follow a Gaussian distribution, with a known mean value and standard deviation. However, other inputs, such as wind conditions or system failures, may behave differently and must receive special treatment. Other inputs that need special treatment are the aerodynamic coefficient curves, which can be sampled by multiplying a curve by a distribution centered at 1.

Valetudo Monte Carlo analysis

An example of a Monte Carlo analysis regarding rocket Valetudo's flight is now presented. In order to investigate RocketPy performance, 20000 simulations were performed, and the evolution of the mean and standard deviation of each output was monitored as a function of the number of simulations/iterations. The means and standard deviations of the simulation parameters employed in this analysis are shown in Table 4.

TABLE 4. Valetudo parameters and their distributions.

Input parameter	μ	σ	Units
Rocket Mass	8.257	0.001	kg
Inertia I	3.67500	0.03675	kg m ²
Inertia Z	0.00700	0.00007	kg m ²
Rocket Radius	40.450	0.001	m
Drag curve multiplier	100.0	3.0	%
Lengths multiplier	100.0	0.1	%
Parachute $C_d S$	0.45	0.07	m ²
Parachute opening delay	1.73	0.52	s
Motor Total Impulse	1415.15	35.30	N s
Burn time	5.274	1.000	s
Nozzle exit radius	21.642	0.500	mm
Nozzle throat radius	8.0	0.5	mm
Grain separation distance	6	1	mm
Grain outer radius	21.400	0.375	mm
Grain inner radius	9.650	0.375	mm
Grain height	120	1	mm
Grain density	1707	50	kg m ⁻³
Launch rail heading	53	2	°
Launch rail inclination	84.7	1.0	°
Launch rail length	5.7000	0.0005	m
Wind profile	ECMWF Ensemble		-

Figure 7 shows the ellipses of impact predicted by RocketPy; all ellipses were evaluated based on the method presented by Chew (1966). The real impact point, measured just after the rocket landed, is also shown and it is represented by a cross plotted at point (411.89 m, -61.07 m). It is possible to see that the real impact point falls into the 3σ ellipse calculated from RocketPy simulations.

Multivariate Rejection Sampling

Running Monte Carlo analyses can be time-consuming. Nonetheless, the expected distribution of simulation inputs can vary, specially during launch operations, requiring new analyses. For example, hours or even minutes before launch, more accurate weather data may become available. Performing a new Monte Carlo analysis considering the new input distributions could take significant time. To solve this, we propose a new method that can run in just a fraction of a second.

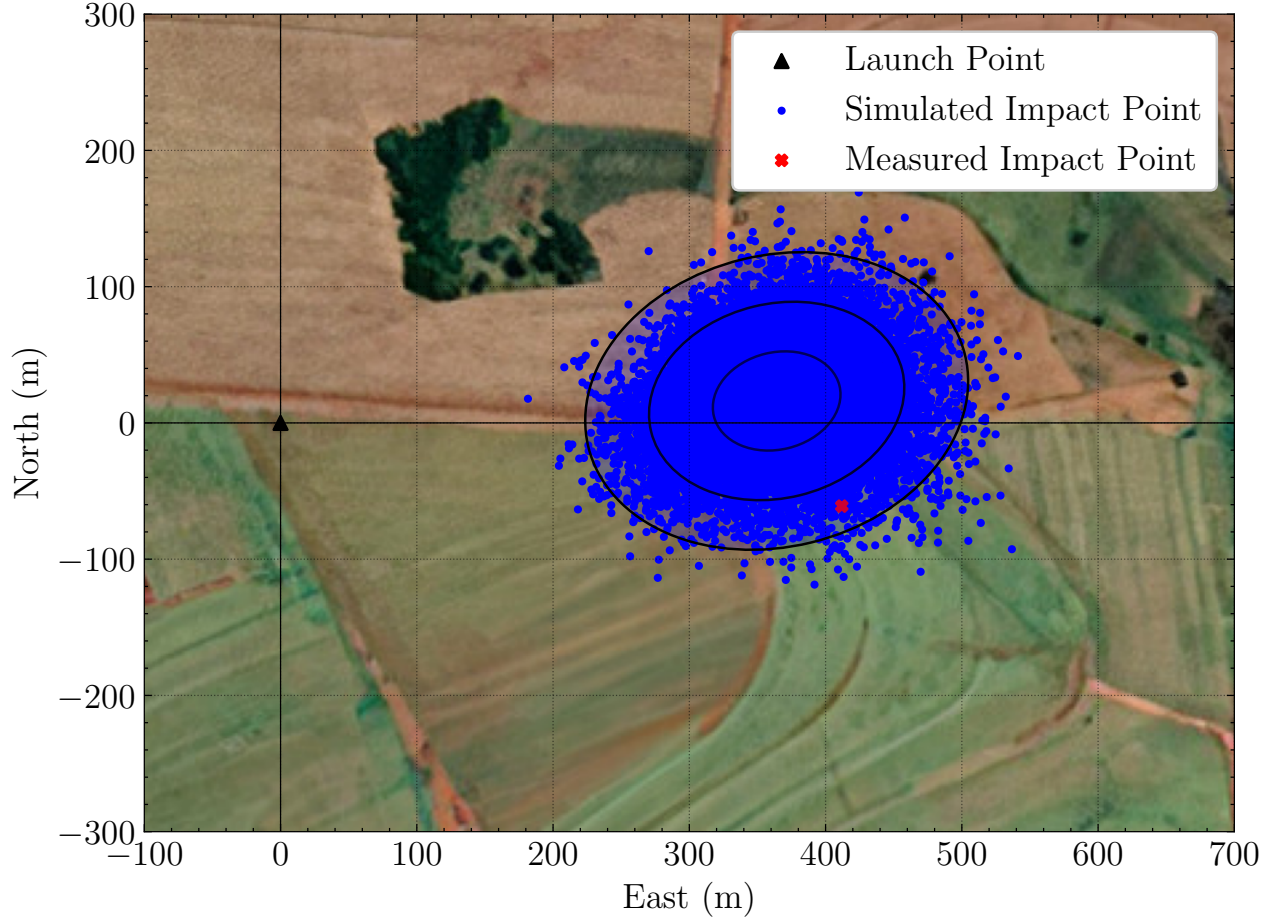


Fig. 7. 1σ , 2σ and 3σ dispersion ellipses of the Impact Point (Image © 2020 Maxar Technologies © 2020 Google).

By extending rejection sampling (Casella et al. 2004) to many variables, it is possible to filter data from a previously run analysis, considering only simulations which would satisfy new input distributions specified by the user. As a consequence, the simulation outputs are correspondingly filtered, configuring a new Monte Carlo analysis. Even though multivariate rejection sampling (MRS) reduces the amount of data available, hence widening confidence intervals, it allows for a rapid solution, useful for informed and fast decision making during launch operations.

To understand the algorithm behind MRS, consider that results from a Monte Carlo analysis $\mathcal{O}$ are available, while a new analysis $\mathcal{N}$ with slightly different input distributions is desired. Both analysis have the same input variables, denoted by X_i , such as rocket mass or launch heading. The probability density functions of the input variables in analysis $\mathcal{O}$, denoted by $f_{X_i}^{\mathcal{O}}$ for each input X_i ,

and the probability density functions of the new desired distributions $f_{X_i}^N$, are needed and generally available. The algorithm loops through every simulation j ran in analysis O , retrieving the sampled values of the input variables, denoted by $x_{j,i}$. Denoting the total number of input variables by n_{in} , the probability of rejecting each simulation j is calculated by:

$$P_{\text{rejection},j} = 1 - \prod_{i=1}^{n_{in}} \left(M(X_i) \frac{f_{X_i}^N(x_i)}{f_{X_i}^O(x_i)} \right), \quad M(X_i) = \sup_x \frac{f_{X_i}^N(x)}{f_{X_i}^O(x)} \quad (14)$$

Given a simulation j in analysis O , a decision is made to determine if such simulation should be included in analysis N or not, respecting the rejection probability $P_{\text{rejection},j}$. Together, the simulations which were accepted form a new Monte Carlo analysis whose input variables closely follow the probability density functions $f_{X_i}^N$, even though all of them came from analysis O .

As an example, consider the graphs in Figure 8. In this case, two input distributions were modified in relation to previously run analyses. Launch rail inclination changed from a normal distribution of $\mu = 84.7^\circ$ and $\sigma = 1^\circ$ to a normal distribution with a different expected value $\mu = 85.2^\circ$, but the same standard deviation $\sigma = 1^\circ$. Launch rail heading, on the other hand, was altered from a normal distribution of $\mu = 53^\circ$ and $\sigma = 2^\circ$ to a normal distribution with the same expected value but smaller standard deviation, $\sigma = 1.5^\circ$. As such, the blue histograms in Figure 8 show the original distributions from previous analyses, while the green histograms overlaid show the resulting distributions after the use of MRS, which correspond to the new input distributions.

Figure 8 also demonstrates how two simulation output distributions, the landing point east and north coordinates, are modified after running MRS. A clear change of expected value and standard deviation is evident when comparing filtered and original data. As expected, given a higher launch rail inclination, the impact coordinates moved closer to the launch site, represented by the origin. In this example, filtering the data from previous Monte Carlo analyses kept 13.6% of the 20000 simulations previously run. However, the confidence interval amplitude for the east landing point coordinate's mean value changed from 0.65 m to 1.73 m, considering 95% confidence calculated using the bootstrap method. This may fall well inside the necessary stopping criteria for a Monte Carlo analysis of this type.

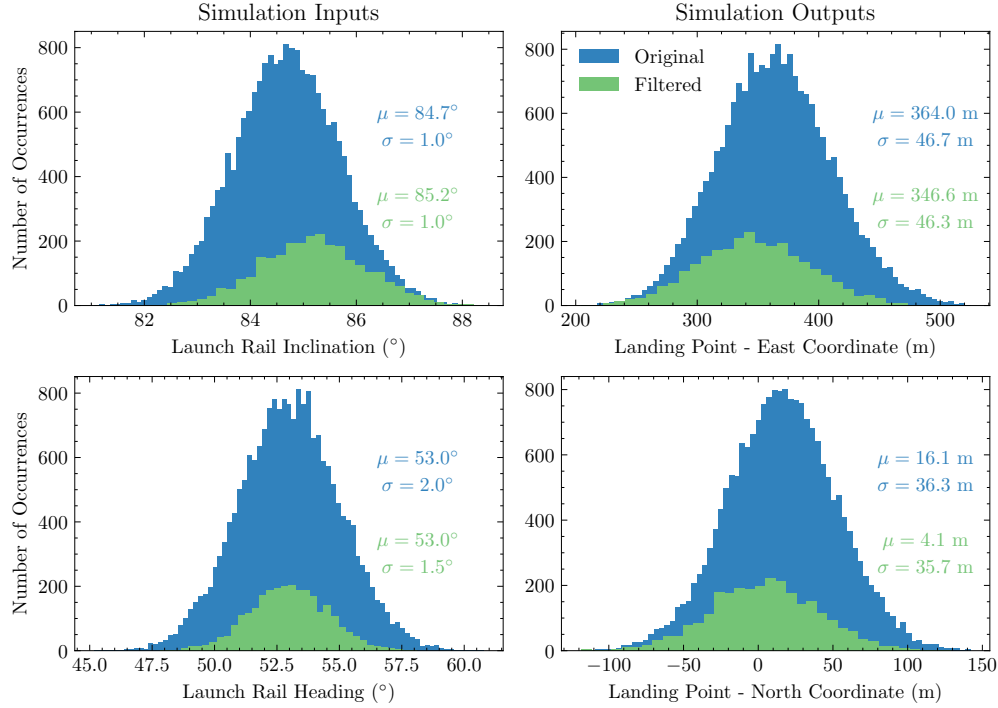


Fig. 8. Application of multivariate rejection sampling.

Finally, it is important to emphasize that Multivariate Rejection Sampling works with inputs behaving with any type of distribution, as long as its probability density function is known, or can be estimated.

Validation and time comparison

To validate the results produced by MRS and evaluate its relative execution time, an equivalent Monte Carlo analysis was run. Such analysis used the same inputs as the MRS example shown above, and also used the same number of simulations. On an Intel(R) Core(TM) i7-2600K CPU @ 3.40 GHz with 16.0 GB of RAM, running Python 3.8.5 under Windows 10 build 19041, applying the MRS algorithm to the original 20000 samples Monte Carlo analysis takes $258 \text{ ms} \pm 57 \text{ ms}$ and produces a new Monte Carlo analysis with an average of 2688 samples. Running the equivalent Monte Carlo analysis, with the same number of samples, takes 1845 s. Therefore, producing new analyses with MRS can be up to 7000 times faster. It is worth noting that both methods, MRS and traditional Monte Carlo, ran in serial mode for this comparison. However, they are both equally

well suited for parallel computing and can, thus, run significantly faster with multiple cores.

Certification that results from MRS and the equivalent Monte Carlo analysis are statistically identical was done using the two samples Kolmogorov-Smirnov test. A minimum p -value of 0.049 was identified when the inputs and outputs distributions from both algorithms were compared, thus, validating the results from MRS.

MRS and Sensitivity Analysis

While the above application example is important during rocket launch campaigns, MRS has a significant potential for sensitivity analysis. In fact, two forms of sensitivity analysis are possible. One, referred in this article as *single simulation sensitivity*, represents the sensitivity of a determined simulation output with respect to a specific simulation input parameter (e.g. the sensitivity of the apogee with respect to the total rocket mass). Another, referred as *Monte Carlo sensitivity*, represents the sensitivity of a Monte Carlo analysis output with respect to a Monte Carlo analysis input (e.g. the sensitivity of the impact dispersion ellipses with respect to the standard deviation of the rocket mass).

Monte Carlo sensitivity can be calculated using the standard sensitivity calculation methods, such as finite differences. MRS presents a significant advantage in this case since it allows for the computation of many new Monte Carlo analysis up to 10^3 times faster, as previously discussed. Without MRS, there would be a need to run various Monte Carlo analysis to perform a detailed sensitivity study, which is often impractical due to the prohibitive time of each analysis. On the other hand, *Single simulation sensitivity* can be calculated in an unusual manner by taking advantage of a Monte Carlo analysis and MRS. Employing the well-known formula for propagation of errors and uncertainties, MRS can be used to generate input and output data with varying standard deviations, presenting a way to calculate sensitivity based on statistical analysis.

CONCLUSION

This paper presented the birth of a powerful high-fidelity 6-DOF vehicle trajectory simulator software. The tool was tested against real flight data retrieved from two high-powered rockets and performed well, thus proving to be effective and reliable. Furthermore, comparisons against already

established software not only endorsed predictions with great agreement between simulations, but established a new frontier of results not yet returned by state of the art programs.

Additionally, the current software is highly flexible, consequently allowing several parameters to be varied as to produce reliable predictions. Previous works already enabled variability of key parameters, but not to the extent presented here. We have shown the applicability of the software for uncertainty quantification by running a dispersion analysis using the Monte Carlo method. Most notably, the present work also introduced the Multivariate Rejection Sampling as a valuable statistical tool with the goal of reducing time spent running Monte Carlo simulations under certain conditions.

It is worth noting that flight data from small sized high-powered rockets was used. Future work should include further testing and validation within the context of larger sounding rockets as to push the boundaries of the software. Additionally, RocketPy serves as an ideal framework for the implementation of new methods related to launch vehicle analysis. Concerning the currently supported motors, hybrid and liquid motors may also be implemented. Regarding aerodynamic models, other methods beyond Barrowman's analysis may be left as an option to the user. Finally, more complex gravitational models can be implemented to enable trajectory design for orbital rockets that require greater regard to perturbative effects.

DATA AVAILABILITY STATEMENT

Some or all data, models, or code generated or used during the study are available in an online repository, which can be accessed in <https://doi.org/10.5281/zenodo.4279966> (Alves et al. 2020), in accordance with funder data retention policies.

The repository holds both the flight data for the experimental rocket missions used in this work (Valetudo, Bella Lui Kaltbrunn and NDRT vehicle) and the corresponding simulation code, including necessary input files, presented in Figure 5 and in Table 3. Furthermore, all models, code and data used for the comparison with previous works presented in Figure 4 and in Table 2 is also included. The Monte Carlo dispersion analysis model and output data is also available, along with the code necessary to generate Figure 7. Finally, the repository also contains the full

implementation of the Multivariate Rejection Sampling algorithm and its application, as shown in Figure 8.

ACKNOWLEDGMENTS

The authors of this work would like to thank the Notre Dame Rocket Team and the École Polytechnique Fédérale de Lausanne Rocket Team for granting access to their data. Moreover, the authors are also thankful for Escola Politécnica of the University of São Paulo, for the opportunity to develop the current publication, and also grateful to all the members of Projeto Jupiter that contributed towards making RocketPy the current robust software. Finally, the members of Projeto Jupiter are especially thankful for the financial aid provided by Amigos da Poli (ADP), whose support is invaluable for the team and consequently for the present paper. BSC also acknowledges financial support from the Brazilian National Council for Scientific and Technological Development (CNPq) in the form of productivity grant (grant number 312951/2018-3).

REFERENCES

- Aitoumeziane, A., Eusebio, P., Hayes, C., Ramachandran, V., Smith, J., Sridharan, J., St Regis, L., Stephenson, M., Tewksbury, N., Tran, M., and Yang, H. (2019). “Traveler IV Apogee Analysis.” *Report no.*, USC Rocket Propulsion Laboratory, Los Angeles, <<http://www.uscrpl.com/s/Traveler-IV-Whitepaper>>.
- Alves, G. F., Ceotto, G. H., Schmitt, R. N., and Pezente, L. A. (2020). “Projeto-Jupiter/RocketPaper, <<https://doi.org/10.5281/zenodo.4279966>> (11).
- Apogee Rockets (2020). “RockSim Information, <<https://www.apogeerockets.com/RockSim/>>.
- Astos Solutions GmbH (2018). “ASTOS - Analysis, Simulation and Trajectory Optimization Software for Space Applications, <<https://www.astos.de/products/astos>>.
- Barrowman, J. S. (1967). “The Practical Calculation of the Aerodynamic Characteristics of Slender Finned Vehicles.” Ph.D. thesis, Catholic University of America, Washington, DC United States.
- Box, S., Bishop, C. M., and Hunt, H. (2011). “Stochastic six-degree-of-freedom flight simulator for passively controlled high-power rockets.” *Journal of Aerospace Engineering*.

- Casella, G., Robert, C. P., and Wells, M. T. (2004). “Generalized Accept-Reject sampling schemes.” *Notes-Monograph Series*, Vol. 45, Institute of Mathematical Statistics, 342–347.
- Chew, V. (1966). “Confidence, Prediction, and Tolerance Regions for the Multivariate Normal Distribution.” *Journal of the American Statistical Association*, 61(315).
- Da Silveira, G. and Carrara, V. (2015). “A six degrees-of-freedom flight dynamics simulation tool of launch vehicles.” *Journal of Aerospace Technology and Management*.
- Eerland, W. J., Box, S., and Sóbester, A. (2017). “Cambridge Rocketry Simulator A Stochastic Six-Degrees-of-Freedom Rocket Flight Simulator.” *Journal of Open Research Software*, 5.
- Eke, F. O. and Mao, T. C. (2002). “On the dynamics of variable mass systems.” *International Journal of Mechanical Engineering Education*, 30(2), 123–137.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). “Array programming with NumPy, <<https://doi.org/10.1038/s41586-020-2649-2>> (9).
- Hindmarsh, A. (1992). “ODEPACK. A Collection of ODE System Solvers.” *Report No. ESTSC-000166CY00100; NESC-9935*, Lawrence Livermore National Lab., CA (United States) (1).
- ISO Central Secretary (1975). “Standard Atmosphere.” *Report No. ISO 2533:1975*, International Organization for Standardization, Geneva, CH (5).
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., and Willing, C. (2016). “Jupyter Notebooks – a publishing format for reproducible computational workflows.” *Positioning and Power in Academic Publishing: Players, Agents and Agendas - Proceedings of the 20th International Conference on Electronic Publishing, ELPUB 2016*, IOS Press BV, 87–90.
- Niskanen, S. (2009). “Development of an Open Source model rocket simulation software.” Ph.D. thesis, Helsinki University of Technology, Espoo (5).
- Notre Dame Rocketry Team (2020). “Flight Readiness Review - Nasa Student Launch 2020 - Lunar

Sample Retrieval System and Air Braking System.” *Report no.*, University of Notre Dame, Notre Dame, IN, <<https://ndrocketry.weebly.com/reports.html>> (3).

Panicucci, P., Allard, C., and Schaub, H. (2018). “Spacecraft Dynamics Employing a General Multi-tank and Multi-thruster Mass Depletion Formulation.” *Journal of the Astronautical Sciences*, 65(4), 423–447.

Petzold, L. (1983). “Automatic Selection of Methods for Solving Stiff and Nonstiff Systems of Ordinary Differential Equations.” *SIAM Journal on Scientific and Statistical Computing*, 4(1), 136–148.

Rogers, C. E. and Cooper, D. (2019). “Rogers Aeroscience RASAero II Aerodynamic Analysis and Flight Simulation Program - User Manual.

Thomson, W. T. (1966). “Equations of motion for the variable mass system.” *AIAA Journal*, 4(4), 766–768.

Thomson, W. T. and Reiter, G. S. (1965). “Jet damping of a solid rocket - theory and flight results.” *AIAA Journal*, 3(3), 413–417.

Van Rossum, G. and Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., and SciPy 1.0 Contributors (2020). “SciPy 1.0: fundamental algorithms for scientific computing in Python.” *Nature Methods*, 17(3), 261–272.

Wilde, P. D. (2018). “Range safety requirements and methods for sounding rocket launches.” *Journal of Space Safety Engineering*, 5(1), 14–21.